The GIL, Not the Syscall: What Batching Buys a Python `io_uring` Runtime on Multiple Cores

Marimuthu Velayutham*

October 6, 2026

Abstract

Python event loops built on Linux `io_uring` report throughput gains over `asyncio` and attribute them to system-call amortization: one `io_uring_enter` reaps many completions. We show that for a multi-threaded Python server the system calls saved are the lesser benefit of batching; the greater one is that CPython's Global Interpreter Lock (GIL) changes hands less often. With one worker, `io_uring` is 1.14–1.25× faster than `epoll`, and batching completions is worth about 20%, all of it obtained by 16 completions per call. With four worker threads in one interpreter on a four-core server, the outcome is decided by how often the GIL is handed over per request. A Python loop on `epoll`, which releases the GIL around two system calls per request, falls to 0.64× the throughput of one worker. The same loop on `io_uring`, which releases it twice per batch, scales by 2.21×, close to loops written in C (2.34× and 2.41×); it does not scale at all (0.98×) if it merely holds the GIL during its submitting system call. We present `uringpy`, a runtime that reaps completions in C and can run the whole accept/receive/send protocol without the GIL. On a standard CPython 3.14 build its worker threads perform as worker processes do (328 thousand requests per second at four workers), where `asyncio` and `uvloop` threads fall to 0.36–0.59× of one worker, and it is 1.33× faster than the best process-per-worker baseline. When every request calls a Python handler, taking the GIL once per request limits its threads to 1.55×, with a measured wait of 8.5 μs per request for the lock; taking it once per batch of requests restores 2.12× and exceeds the best process-mode baseline by 1.25×. A cost model bounds thread scaling by the share of a request spent under the GIL; the bound holds in every configuration measured and is tight for handlers of 30 μs or more. On the free-threaded build `asyncio` threads scale again and the runtime's advantage is a factor of 1.5–2.0 in speed, not a difference in scaling. Every configuration is measured in five interleaved repetitions and reported with 95% confidence intervals; the code, the raw runs and the script that generates the tables are public.

Index Terms— io_uring, asynchronous I/O, Python, Global Interpreter Lock, event loop, multicore scaling, system-call amortization

*M. Velayutham is an independent researcher (e-mail: marimuthu.vm@gmail.com; ORCID: 0009-0007-7352-5552).

1 Introduction

Completion-based kernel interfaces such as Linux `io_uring` [1] let an application submit many I/O operations and reap many completions with a single system call, in contrast to readiness-based interfaces (`epoll`), which report which sockets are ready and leave the application to make one system call for every read and every write. A growing set of Python projects wrap `io_uring` behind an `asyncio`-compatible event loop and report throughput improvements over the standard library loop and over `uvloop` [2], generally crediting the win to system-call amortization.

Two explanations for such a win are possible, and they are rarely separated. One is the interface: fewer system calls per request. The other is where the per-event loop runs. CPython serializes bytecode execution behind a Global Interpreter Lock (GIL), so a loop that dispatches every event in Python pays an interpreter cost per event and, when several worker threads run such loops, lets only one of them proceed at a time. A faster interface does nothing about that cost, and a loop moved out of the interpreter would be faster on any interface. Crediting a measured gain to `io_uring` therefore needs an experiment in which the two are varied independently.

We run that experiment. One server loop is implemented four times: on `io_uring` and on `epoll`, each in C with the GIL released and in Python. For a single worker both factors matter, modestly: `io_uring` is 1.14–1.25× faster than `epoll`, a loop in C is 1.05–1.15× faster than the same loop in Python, and restricting `io_uring` to one completion per system call costs 20%. For worker threads sharing one interpreter, neither factor decides the outcome by itself. What decides it is how often the GIL changes hands. A Python loop on `epoll` gives up the GIL at each of its two system calls per request, and four such workers deliver 0.64× the throughput of one. The same loop on `io_uring` gives it up twice per batch of completions, and four workers deliver 2.21×, within 10% of the loop written in C. The same loop again, changed only to keep the GIL during its one submitting call, delivers 0.98×. Batching matters to a multi-threaded Python server chiefly because it batches GIL hand-offs; the sys-

tem calls it saves are the smaller gain. That is the sense of the title.

Guided by this, we build a runtime, `uringpy`, that reaps completions in C and offers three ways of keeping hand-offs rare: a reactor that runs the full accept/receive/send state machine inside one `nogil` region and executes no Python at all; the same reactor calling a Python handler, with the GIL taken once per batch of requests instead of once per request; and a batch interface on which a Python loop can be written. N workers, each with its own ring and its own `SO_REUSEPORT` listener, scale across cores inside a single interpreter on a standard GIL build. We also measure where the advantage ends: on the free-threaded build, where `asyncio` threads scale again; when the handler is long enough for the GIL itself, not its hand-offs, to be the limit; and when responses are large enough for the network to be the limit.

Contributions.

- A cost model and controlled experiments—a contention probe, a 2×2 design that crosses the system-call interface with the placement of the loop, a batch-size cap, and a handler-cost sweep with measured GIL hold and wait times—showing that the interface sets part of single-worker throughput, while thread scaling is governed by the time a request spends under the GIL and by the number of GIL hand-offs it causes (Sections 3 and 5).
- A GIL-aware runtime that reaps completions and runs the connection protocol in C, and that takes the GIL once per batch of requests when a Python handler is called, raising thread scaling with a handler from 1.55× to 2.12× (Section 4).
- An evaluation across the thread/process model × GIL/free-threaded build matrix—spanning transport-only and application-handler workloads and batching and response-size regimes, with confidence intervals and processor accounting—including negative and boundary results (Section 5).

2 Background and Related Work

Amortizing system calls. The cost of crossing the user/kernel boundary once per I/O operation has long been a target. FlexSC [3] showed that batching system calls and executing them asynchronously removes much of the direct and indirect cost of mode switches. MegaPipe [4] combined system-call batching with partitioned listening sockets and lightweight connection handles, and user-level or dataplane stacks such as mTCP [5] and IX [6] batch events and packets aggressively. `io_uring` [1] brings the same idea into the mainline Linux interface: a submission ring and a completion ring are shared between user space and the kernel, and one `io_uring_enter` call submits many operations and reaps many completions. Studies of `io_uring` have so far concentrated on storage and database engines. Didona et al. [7] compare it with `libaio` and SPDK and find that its performance depends strongly on how it is configured, and Jasny et al. [8] report that substituting it for existing I/O calls gains little in a database system, whereas a design built around its asynchronous, batched model gains substantially. Both results warn against attributing a speed-up to the interface alone; this paper asks the same question for a managed-language network server, where a second cost—the interpreter—is present on every event.

Event-driven and shared-nothing servers. Event-driven servers built on readiness notification [9,10] are the model that `asyncio` inherits [11]. Scaling such servers across cores is usually done by giving each core its own event loop and its own accept queue. Affinity-Accept [12] showed the benefit of per-core accept queues for connection locality, `SO_REUSEPORT` [13] made the kernel distribute connections across several listening sockets, and thread-per-core designs have been studied for their effect on tail latency [14]. Our workers follow this structure: one ring and one `SO_REUSEPORT` listener per worker, with no state shared between them.

The GIL. CPython serializes bytecode execution behind a Global Interpreter Lock. Beazley [15] characterized how hand-off of the lock between threads behaves, including the degradation that I/O-bound threads suffer when other threads compete for it. Three lines of work aim to remove the limitation. The lock can be made unnecessary inside the interpreter, as explored for Ruby with hardware transactional memory [16] and now delivered for CPython by the free-threaded build [17], which is a supported configuration from Python 3.14 [18]. The lock can be multiplied: each subinterpreter has had its own GIL since Python 3.12 [19], with a standard-library interface in 3.14 [20]. Or the work can be moved out from under the lock, into native code that releases it; Cython [21] makes this convenient through `nogil` regions, and it is the approach taken here. On free-threaded builds the standard event loop itself has been reworked to scale across threads [22], which is why we evaluate against CPython 3.14 with and without the GIL. Whatever part of a request still runs under the lock bounds thread scaling in the manner of Amdahl's argument [23]; Section 3 makes this bound explicit.

Python servers and event loops. `uvloop` [2] reimplements the `asyncio` loop over libuv. Several projects (e.g. `uringloop` [24], `uringcore` [25]) provide `io_uring`-backed loops and attribute their gains over `asyncio` to fewer system calls; like `asyncio`, they dispatch every event to Python code. Moving network I/O out of the interpreter is itself established practice: Granian [26], for example, performs it on Rust threads built on Hyper and Tokio and enters Python only to run the application, and deployments under the GIL conventionally run one process per core [27, 28]. Completion-based network I/O is likewise established outside Python [29, 30].

Our runtime applies the principle to `io_uring` completion reaping in Python. It is also deliberately small enough to be paired with an otherwise identical `epoll` reactor and with Python-dispatch loops on each interface. That pairing supplies what has been missing: a measurement that separates the candidate explanations for the speed of such designs—the system-call interface, the placement of the per-event loop, and the handling of the GIL—and that measures what happens to thread scaling in one interpreter once every request enters Python. We did not measure the `io_uring` loops cited above, so we make no claim about which regime each of them is in.

Measurement practice. Performance comparisons are easily distorted by too few repetitions and by uncontrolled properties of the setup [31,32], and cloud networks add variability of their own [33]. We therefore repeat every configuration, interleave the repetitions, and report means with confidence intervals together with CPU use on both machines. Load is generated with `wrk` [34], a closed-loop generator; its latency figures describe latency at the offered load and are subject to coordinated omission [35], so we use them only to compare configurations at the same load, not as tail-latency estimates.

3 Isolating the Bottleneck

Hypotheses. We began from a broad hypothesis: that for a Python `io_uring` server, where the per-event loop runs, in C or in Python, matters more than which system-call interface is used. The experiments of Section 5 do not support it. With one worker the two matter about equally, and with several workers a loop in Python scales nearly as well as a loop in C, provided it seldom gives up the GIL. What the measurements do support, and what the rest of the paper examines, is a statement about the lock itself: the scaling of worker threads is governed by how much of each request runs under the GIL and by how often the GIL changes hands per request. The system-call interface matters for scaling through the second of these, because an interface that batches completions also lets a loop batch its hand-offs.

Cost model. Let a request cost t_c of processor time that needs no interpreter state, and so can run with the GIL released, and t_p that must run with the GIL held. A saturated single worker serves $X_1 = 1/(t_c + t_p)$ requests per second. With N worker threads in one interpreter on at least N cores, two constraints apply. Each worker still needs $t_c + t_p$ per request, so $X_N \le N/(t_c + t_p)$. And only one thread holds the GIL at a time, so the GIL is busy for $X_N\, t_p \le 1$ of every second, that is, $X_N \le 1/t_p$. Dividing by X_1 gives the scaling bound

$$S(N) = \frac{X_N}{X_1} \le \min\left(N,\ \frac{1}{f}\right), \qquad f = \frac{t_p}{t_c + t_p}, \qquad (1)$$

where f is the fraction of a request spent holding the GIL. A loop that holds the GIL for all of a request's processor time has $f = 1$ and cannot exceed one worker's throughput however many threads it runs. A loop that never enters the interpreter has $f = 0$ and is bounded only by N. A loop written in Python need not be near $f = 1$: system calls made with the GIL released count towards t_c. With one process per worker each worker has its own GIL and the second constraint disappears, leaving $S(N) \le N$ for any f; process-mode results are thus the reference for what the hardware allows.

Hand-offs. The bound (1) counts only the time the GIL is held. It ignores what it costs to pass the lock from one thread to another: the releasing thread must wake a waiter, and the waiter must be scheduled before it can run. Models of contended critical sections account for such costs [36]. Let a request cause h hand-offs, and let each cost c when another thread is waiting. The GIL is then unavailable for $X_N\,(t_p + h\,c)$ of every second, so

$$X_N \le \frac{1}{t_p + h\,c}, \qquad S(N) \le \min\left(N,\ \frac{1}{f + h\,c\,X_1}\right). \quad (2)$$

The count h is a property of the design, not of the request. A loop that releases the GIL around every system call has h equal to its system calls per request. A C reactor that takes the GIL for each handler call has $h = 1$. A loop that gives up the GIL once per batch of B requests has h of the order of $1/B$. This is where the system-call interface enters: a completion interface that returns many completions per call lets a loop release the GIL per batch, whereas a readiness loop that makes one system call per read and per write releases it at each of them.

We measure all the quantities involved: h, by counting GIL releases or acquisitions; t_p and the time w a request waits to acquire the GIL, by timing each handler call; and from them the *GIL utilisation* $U_N = X_N\, t_p \le 1$, the share of each second in which the GIL is held running request code. What is left, $1 - U_N$, is consumed by hand-offs or is idle.

Method. We process a fixed number of synthetic completions two ways with identical per-completion arithmetic (a `splitmix64` step), while K CPU-bound Python threads contend for the GIL:

- *reactor_python*: an interpreted per-completion loop (GIL held), modeling per-event Python dispatch as in `asyncio`.
- *reactor_nogil*: the same arithmetic inside one C `nogil` region (GIL released), modeling C-side batch reaping.

Each design is auto-calibrated to run ≈0.25 s at rest, so both are measured over comparable, stable durations. We report each design's *contention slowdown* $= t(K)/t(0)$.

Result. With one competing thread the interpreted loop takes 2.04 times as long and the C loop 1.02 times, with standard deviations below 1% (Table 1). With two and three competitors the timings scatter widely—the standard deviation of the Python loop's time reaches half its mean—and the C loop is affected as well: its fastest

Table 1: Contention probe: time in seconds to process a fixed number of events while K threads compete for the GIL, as mean (standard deviation) over 20 repetitions, with the slowdown relative to $K = 0$ computed from the means and from the fastest repetition (min).

	Python loop			C `nogil` loop		
K	time (sd)	mean	min	time (sd)	mean	min
0	0.251 (0.001)	1.00	1.00	0.250 (0.000)	1.00	1.00
1	0.512 (0.003)	2.04	2.03	0.255 (0.000)	1.02	1.02
2	0.866 (0.278)	3.46	2.09	0.413 (0.166)	1.65	1.00
3	1.137 (0.588)	4.54	2.17	0.394 (0.086)	1.58	1.03

repetition stays within 3% of the uncontended time, but its mean rises to about 1.6 times. On this four-core machine the competing threads and the measured thread evidently do not always have a core each. We therefore rely on the probe only for $K = 1$: an interpreted loop shares the GIL about equally with one competitor, and a loop that has released the GIL is not slowed by it. The probe performs no I/O and makes no system call, so it says nothing about the interface; Section 5 varies that. (Environment: the server of Table 2, CPython 3.14.7, GIL build.)

4 Design

`uringpy` is a Cython [21] extension over `liburing`. Its unit is the *engine*: one `io_uring` instance, one pool of fixed-size receive buffers, and a reactor loop. A server is N engines, each driven by its own OS thread and each with its own `SO_REUSEPORT` listening socket on the same port, so the kernel spreads incoming connections over the workers (Fig. 1). Engines share nothing in user space: no ring, buffer or connection is touched by two workers, so the data path takes no user-level lock. Each ring is created with default flags and uses single-shot accept, receive and send operations, without registered buffers or files; the interface effects reported in Section 5 are for this basic use of `io_uring`.

The loop. Fig. 2 gives the loop each worker runs; all of it executes inside one `nogil` region. Each iteration makes exactly one system call, `io_uring_submit_and_wait`, which hands the kernel every operation queued while the previous batch was processed and returns when at least one completion is ready. The loop then processes every completion already in the ring without re-entering the kernel. Follow-up operations (the next receive, the send of a response) are written into the submission ring and leave with the next iteration's call. No Python object is created for a completion, and the interpreter is not entered.

Per-operation state. Each submitted operation carries a 64-bit tag that the kernel returns with its completion: 8 bits of operation type, 32 bits of file descriptor, and 24 bits that hold the receive-buffer slot for a receive or the number of response bytes already sent for a send.

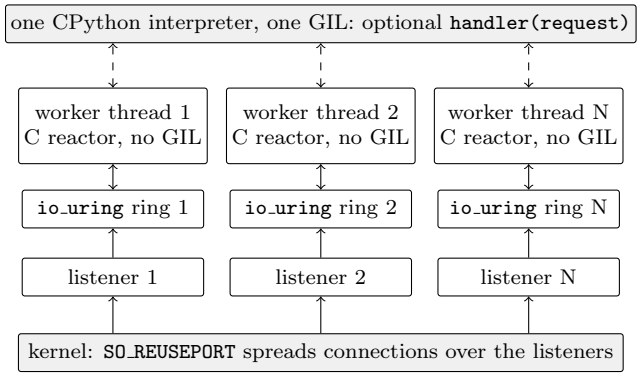


Figure 1: A server is N engines in one process. Solid arrows are the data path, which never leaves C; the dashed arrow is the only point where a worker takes the GIL, and exists only in application mode.

Dispatch therefore needs no lookup table. Because the sent offset travels with the operation, a send that the kernel completes only in part is continued from where it stopped, and a response may be larger than a buffer slot (up to 16 MiB). Receive buffers come from a per-engine slab allocator; a connection for which no buffer or submission entry is available is closed, not left idle.

Two entry points. `serve_forever_echo` answers every received request with a fixed response and never takes the GIL; it measures the transport path alone. `serve_forever_app` passes each request to a Python callable. The reactor takes the GIL for that call only: it builds the request `bytes`, invokes the handler, copies the result into a per-connection buffer owned by the engine, and releases the GIL before the send is queued. Accept, receive and send stay outside the GIL in both. By default the GIL is taken once per request, so each request causes one hand-off when another worker is waiting ($h = 1$ in (2)). With *GIL batching* enabled, the requests found in one pass over the completion queue are collected and the GIL is taken once for all of them: the handler receives the same calls in the same order, but the lock changes hands once per batch instead of once per request, at the price that no response of a batch is queued until all its handlers have run. Neither entry point parses HTTP: each replies once per `recv`, so pipelined and fragmented requests are out of scope. The same holds for every baseline we compare against.

A batch interface for Python. The engine also exposes submission and reaping to Python code: operations are queued without a system call, `flush` submits everything queued, and `get_events` returns the completions that have arrived as one list. Both release the GIL around their system call, so a loop written in Python on this interface gives up the GIL twice per batch however many requests the batch contains. The kernel can perform queued operations, a send for instance, inside the submitting call; whether the GIL is held across that call therefore decides how much of a request's processor time

```
1: queue ACCEPT on the listener
2: while not stopped do
3:   io_uring_submit_and_wait(1) {one system call}
4:   for each completion c in the ring, up to the batch
     cap do
5:     if c is an ACCEPT then
6:       queue ACCEPT; queue RECV on the new con-
         nection
7:     else if c is a RECV of n > 0 bytes then
8:       if application mode then
9:         take GIL; r ← handler(request)
10:        copy r to the connection's buffer; release
           GIL
11:      end if
12:      queue SEND of the response from offset 0
13:    else if c is a SEND of n > 0 bytes then
14:      if part of the response is still unsent then
15:        queue SEND of the remainder
16:      else
17:        count one request; queue RECV
18:      end if
19:    else
20:      close the connection {peer closed, or error}
21:    end if
22:  end for
23: end while
```

Figure 2: The reactor loop of one worker. Queued operations reach the kernel at the next execution of line 3.

is spent under it.

Instrumentation. The reactor counts its `io_uring_enter` calls, its completions and the responses it has sent in full, so system calls per request and completions per system call are measured, not estimated. A batch cap limits the completions processed per system call, to vary batching in isolation. GIL acquisitions made for handler calls are counted, and optional timing records, per handler call, how long the worker waited for the GIL and how long it held it. The counters are plain integer increments; the timing is disabled unless requested.

A readiness-based twin. For the comparison in Section 5 the package also contains `EpollEngine`, the same loop written against `epoll`: one `recv` and one `send` system call per request, the same reply-once-per-`recv` behaviour and resumption of partial sends, likewise inside one `nogil` region, with every system call counted.

Interpreter integration. A wait that a signal interrupts returns control to the interpreter long enough to run pending Python signal handlers, so a worker on the main thread can be stopped normally. The extension declares itself safe for free-threaded builds, so loading it there does not re-enable the GIL; the benchmark image verifies this when it is built.

Table 2: Testbed.

Server	Google Compute Engine `t2d-standard-4`: 4 vCPUs (AMD EPYC 7B13), one per physical core, 16 GB
Client	`t2d-standard-8`: 8 vCPUs (AMD EPYC 7B13), one per physical core; same zone (`us-east1-b`)
System	Ubuntu 24.04 LTS, Linux 7.0.0-1011-gcp on both
Runtime	Docker, Debian 12 base, `liburing` 2.6; CPython 3.14.7 built from source with and without `--disable-gil`
Load	`wrk` 4.1.0, 8 threads, 400 keep-alive connections, `GET /`; 13-byte response body unless stated
Run	fresh server process; 5 s warm-up (discarded), 20 s measured
Repetitions	5 per configuration, interleaved in shuffled order

5 Evaluation

5.1 Setup

Table 2 lists the testbed. The server and the load generator are separate virtual machines in one zone. Both are of a machine family in which each vCPU is a whole physical core [37], so a worker count equals a core count, and the client has twice the server's cores so that it is not the limit. The code under test runs in a container on the server with host networking. The two interpreters are built from one recipe that differs only in `--disable-gil`, so a comparison between them is not also a comparison of compilers or build options, and every run records the interpreter version and the GIL state the server process reports.

5.2 Engines

Table 3 lists the servers compared. All of them answer each `recv` with the same bytes and none parses HTTP, so they differ only in how the I/O loop is built and in when the GIL is held. The first four form a 2×2 design that varies the system-call interface and the placement of the per-event loop independently. Each engine runs as N workers with one `SO_REUSEPORT` listener per worker, either as threads of one interpreter (*thread* mode, one GIL) or as separate processes (*process* mode, one GIL each).

The two Python loops of the 2×2 design are bare dispatch loops written directly on the interface, with none of the task, future and transport machinery of `asyncio`; they isolate the cost of running the loop in the interpreter and are not event loops one would deploy. They differ in one respect besides the interface, and it turns out to be the one that matters. `py-epoll` makes its `recv` and `send` calls through CPython's socket objects, which release the GIL around every system call. `py-uring` uses the batch interface of Section 4 and releases the GIL around its two calls per batch. A fifth engine, `py-uring-held`, is

Table 3: Engines. "C" means the loop runs with the GIL released; the handler is the same Python function for every engine that calls one.

Engine	Interface	Per-event loop and GIL
`uringpy`	`io_uring`	C
`c-epoll`	`epoll`	C (same loop)
`py-uring`	`io_uring`	Python; GIL released per batch
`py-epoll`	`epoll`	Python; GIL released per syscall
`py-uring-held`	`io_uring`	as `py-uring`, GIL held in submit
`asyncio`	`epoll`	Python, streams API
`asyncio-proto`	`epoll`	Python, Protocol API
`uvloop(-proto)`	libuv	C loop, Python callbacks
`uringpy-app`	`io_uring`	C + handler, GIL per request
`uringpy-app-batch`	`io_uring`	C + handler, GIL per batch
`asyncio-app`	`epoll`	Python, streams API + handler
`asyncio-proto-app`	`epoll`	Python, Protocol API + handler
`uvloop-proto-app`	libuv	C loop, Protocol API + handler

`py-uring` changed only to keep the GIL while it submits; it is not a cell of the design, and is run beside it to show what that one choice does.

5.3 Metrics

For a configuration with n runs and throughputs $x_1, \ldots, x_n$ (requests per second over the measured interval) we report the mean $\bar{x}$ with the half-width of its 95% confidence interval,

$$h = t_{0.975,\,n-1}\,\frac{s}{\sqrt{n}}, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \quad (3)$$

where t is the Student-t quantile (2.776 for $n = 5$). Scaling at N workers is the ratio to the one-worker configuration of the same engine and mode, with a half-width from first-order error propagation:

$$S_N = \frac{\bar{x}_N}{\bar{x}_1}, \qquad h_S = t\,S_N\sqrt{\frac{s_N^2}{n_N\,\bar{x}_N^2} + \frac{s_1^2}{n_1\,\bar{x}_1^2}}, \quad (4)$$

with t taken at $\min(n_N, n_1) - 1$ degrees of freedom.

Processor use is sampled from `/proc/stat` on both machines immediately before and after each measured interval. With u the busy share of a machine with c cores, $C = c\,u$ is the number of busy cores, and we report throughput per busy core, $\bar{x}/C$, next to S_N: a one-worker server also uses other cores for kernel network processing, so S_N alone understates per-core scaling. We also give this quantity relative to one worker, $E_N = (\bar{x}_N/C_N)/(\bar{x}_1/C_1)$, so that $S_N = E_N\,C_N/C_1$. A configuration is flagged when the client's busy share exceeds 85%, since the load generator may then be the limit.

From the reactor's counters we report system calls per request and completions per `io_uring_enter`, each as a ratio of counters summed over a configuration's runs, and, in application mode, the GIL hold time t_p and wait time w per handler call. Latency percentiles are those of `wrk` and are used only to compare configurations at equal load (Section 2).

5.4 Criterion

The broader hypothesis of Section 3 was given a test. It is supported if, in the 2×2 design, moving the loop from Python to C changes throughput far more than switching between `epoll` and `io_uring`, and if capping the completions processed per system call changes throughput little. Otherwise the paper reports what the data shows. We do not present this as a pre-registered test: the criterion was written down on the day of the measurements, while the 2×2 runs were in progress, and the commit that records it is later than those runs.

5.5 Results

The result tables are generated from the raw runs by a script in the repository (`benchmarks/make_tables.py`); no number in them is transcribed by hand. They cover 1135 runs of 227 configurations, and no run failed. Hypervisor steal time was zero on both machines throughout. The client's busy share, averaged over a configuration's runs, never exceeded 78%; a single run reached 86%, in a configuration marked for its variation in Table 10. No run reported a non-2xx response. Socket errors were reported only in the handler sweep with 1000 or 3000 iterations of added work, for at most 0.4% of a run's requests.

The 2×2 design. Table 4 gives the four cells and Table 5 the effect of each factor with the other held fixed. The counters confirm that the design varies what it is meant to vary: the `epoll` loops make 2.0 system calls per request (one `recv` and one `send`), the `io_uring` loops between 0.006 and 0.042.

With one worker, both factors matter and neither dominates. Changing the interface from `epoll` to `io_uring` raises throughput by a factor of 1.14 ± 0.04 when the loop is in C and 1.25 ± 0.03 when it is in Python. Moving the loop from Python to C raises it by 1.05 ± 0.04 on `io_uring` and 1.15 ± 0.03 on `epoll`. The second pair is a lower bound on what a full event loop pays, because the Python loops here are bare: `asyncio`'s streams API, which like `py-epoll` runs on `epoll`, reaches 66.8 thousand requests per second on one worker (Table 8) against 106.8 for `py-epoll`.

With four workers the result is not the one we set out to show. Both C loops scale, to $S_4 = 2.34 \pm 0.07$ on `io_uring` and 2.41 ± 0.06 on `epoll`. So does the Python loop on `io_uring`: $S_4 = 2.21 \pm 0.04$, at 295.1 thousand requests per second, which is 0.90 of the C loop on the same interface and 0.99 of the C loop on `epoll`. The Python loop on `epoll` does not: it gains at two workers (1.32 ± 0.03) and falls below one worker at four (0.64 ± 0.01). The placement of the loop therefore does not decide scaling. On `io_uring`, moving the loop from Python to C is worth 1.05× at one worker and 1.11× at four. What separates the two Python loops is the interface, worth 1.25× at one worker and 4.33× at four.

Why the interface matters to threads. The reason is not the system calls saved: `c-epoll`, which makes

Table 4: The 2×2 design (worker threads, GIL build, 13-byte body): throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, measured system calls per request at one and four workers, busy server cores and 99th-percentile latency at four workers.

Engine	Interface	Loop	N=1	N=2	N=4	S_2	S_4	sys/req$_1$	sys/req$_4$	C_4	p99$_4$ (ms)
`uringpy`	`io_uring`	C	140.3 ± 4.3	230.6 ± 6.9	329.0 ± 0.8	1.64 ± 0.07	2.34 ± 0.07	0.006	0.022	3.92	1.8
`c-epoll`	`epoll`	C	123.4 ± 2.9	200.5 ± 3.8	297.7 ± 0.8	1.62 ± 0.05	2.41 ± 0.06	2.004	2.014	3.92	2.8
`py-uring`	`io_uring`	Python	133.5 ± 2.6	212.4 ± 2.9	295.1 ± 0.7	1.59 ± 0.04	2.21 ± 0.04	0.011	0.042	3.81	1.9
`py-epoll`	`epoll`	Python	106.8 ± 1.8	141.6 ± 2.7	68.1 ± 0.9	1.32 ± 0.03	0.64 ± 0.01	2.004	2.012	2.46	11.5
`py-uring-held`	`io_uring`	Python‡	131.5 ± 3.6	129.1 ± 2.8	128.6 ± 2.3	0.98 ± 0.03	0.98 ± 0.03	0.011	0.040	1.51	783.7

‡ `py-uring` holding the GIL during the submitting system call; shown for comparison and not part of the 2×2 design.

Table 5: Effect of each factor with the other held fixed, as a ratio of mean throughputs (± 95% half-width) from Table 4. Interface: `io_uring` over `epoll`. Loop: C over Python.

Effect	N=1	N=2	N=4
Interface, loop in C	1.14 ± 0.04	1.15 ± 0.04	1.11 ± <0.01
Interface, loop in Python	1.25 ± 0.03	1.50 ± 0.03	4.33 ± 0.06
Loop, on `io_uring`	1.05 ± 0.04	1.09 ± 0.04	1.11 ± <0.01
Loop, on `epoll`	1.15 ± 0.03	1.42 ± 0.04	4.37 ± 0.06

two per request, scales best of all. It is the GIL. `py-epoll` releases the GIL around each of its system calls, so the lock changes hands about twice per request; we count 2.012 releases per request. `py-uring` releases it around one submitting and one waiting call per batch, which at four workers is 0.042 releases per request. Table 6 orders every configuration of this section by that count. Those that give up the GIL less often than once in twenty requests scale by 2.1–2.4×, whether the loop is written in C or in Python and whether or not Python code runs for each request. The reactor that takes the GIL once per request scales by 1.55×. The loop that releases it twice per request falls to 0.64×. This is the ordering that (2) predicts from h.

The last row of the table shows that the count is not the whole story, as (1) says it cannot be. `py-uring-held` gives up the GIL as rarely as `py-uring`, but keeps it while submitting, and the kernel performs the queued sends inside that call. Most of a request's processor time is then spent under the GIL, and four workers deliver the throughput of one (0.98 ± 0.03) with 1.51 of the four cores busy; `wrk` reports a 99th-percentile latency near 0.8 s while the median falls to 0.9 ms, so some connections are served quickly and others starve. Releasing the GIL around that one call is the entire difference between 0.98× and 2.21×. An earlier measurement of ours had this behaviour in the `py-uring` cell and led us to conclude that a Python loop cannot scale on either interface; those runs are kept in the repository as superseded.

Batching. Table 7 varies batching alone, on one worker of the C reactor. Capped at one completion per call, `uringpy` makes 2.0 system calls per request, the same number as an `epoll` loop, and delivers 0.80 ± 0.03 of its uncapped throughput. A cap of 4 gives 0.93 ± 0.04, and from a cap of 16 upward throughput cannot be distinguished from the uncapped run, although system calls per request go on falling twentyfold, from 0.126 to 0.006. For a single worker, then, the system calls that batching saves are worth about 20%, and all of that is obtained by 16 completions per call. An uncapped single worker reaches 330–380 completions per call across our experiments, and each of four workers, among which the 400 connections are divided, about 90; both are far beyond what the saving of system calls requires. Capped at one, `io_uring` is somewhat slower than the `epoll` loop (115.3 against 123.4 thousand requests per second, measured in two different experiments): without batching, the completion interface has no advantage here. The larger effect of batching is the one in Table 6, on the number of GIL hand-offs when the loop or the handler is in Python. We did not vary the batch cap for those configurations.

Outcome of the criterion. The criterion of Section 5.4 was written for the broad hypothesis, and it is not met. Its first condition, that moving the loop changes throughput far more than changing the interface, fails at one worker (1.05–1.15× against 1.14–1.25×) and fails at four workers on `io_uring`, where both are worth 1.11×; it holds only on `epoll` (4.37×). Its second condition, that capping the batch changes throughput little, holds for caps of 16 and above and fails below. We therefore claim neither that the system call is unimportant nor that the loop must be in C. The statement that the data supports was formulated after seeing these results and should be read as such: the interface, and batching up to a small batch size, set part of one worker's throughput; whether worker threads scale under the GIL depends on how much of a request runs under it and on how often it is handed over; and batching is the means by which a loop, in C or in Python, hands it over rarely.

Scaling across cores. Table 8 compares `uringpy` with the `asyncio` and `uvloop` baselines on the transport path. Three observations follow.

First, `uringpy` worker threads perform as worker processes do: at four workers the ratio of the two is 0.998 ± 0.004. On this path one interpreter with four threads delivers what four interpreters deliver. That is what one should expect of a path that executes no Python, and the measurement confirms that nothing else in the runtime or the interpreter gets in the way.

Table 6: Thread scaling against GIL hand-offs per request (four worker threads, GIL build, 13-byte body). Hand-offs are measured at four workers: GIL releases per request for the Python loops, GIL acquisitions per request for the C loop with a handler. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$).

Configuration	Engine	Hand-offs/req.	$\bar{x}_1$	$\bar{x}_4$	S_4	C_4
C loop, `io_uring`; no Python	`uringpy`	0	140.3 ± 4.3	329.0 ± 0.8	2.34 ± 0.07	3.92
C loop, `epoll`; no Python	`c-epoll`	0	123.4 ± 2.9	297.7 ± 0.8	2.41 ± 0.06	3.92
C loop, handler, GIL per batch	`uringpy-app-batch`	0.022	130.1 ± 3.5	275.6 ± 5.6	2.12 ± 0.07	3.70
Python loop, `io_uring`	`py-uring`	0.042	133.5 ± 2.6	295.1 ± 0.7	2.21 ± 0.04	3.81
C loop, handler, GIL per request	`uringpy-app`	1.000	126.3 ± 1.7	196.3 ± 4.5	1.55 ± 0.04	3.24
Python loop, `epoll`	`py-epoll`	2.012	106.8 ± 1.8	68.1 ± 0.9	0.64 ± 0.01	2.46
Python loop, `io_uring`, GIL held in submit‡	`py-uring-held`	0.020	131.5 ± 3.6	128.6 ± 2.3	0.98 ± 0.03	1.51

‡ Releases the GIL as rarely as `py-uring`, but the kernel performs the queued sends while the GIL is held, so nearly all of a request's processor time is spent under it.

Table 7: Batch cap (`uringpy`, one worker): completions processed per `io_uring_enter` limited to the cap. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), ratio to the uncapped run, measured completions per call and system calls per request.

Cap	Throughput	vs. none	compl./call	sys/req
1	115.3 ± 3.4	0.80 ± 0.03	1.0	2.001
4	134.2 ± 4.4	0.93 ± 0.04	4.0	0.501
16	143.3 ± 2.5	1.00 ± 0.04	15.9	0.126
64	141.4 ± 4.5	0.98 ± 0.04	63.0	0.032
256	142.7 ± 2.3	0.99 ± 0.03	241.9	0.008
none	144.0 ± 4.5	1	356.1	0.006

Second, the scaling it reaches, $S_4 = 2.33 \pm 0.05$, is what this server allows, not a limit of the runtime. One worker already keeps $C_1 = 1.18$ cores busy, because kernel network processing for its connections runs partly on other cores, and four workers keep 3.91 of the four cores busy. Per busy core, throughput at four workers is $E_4 = 0.70$ of its value at one. The baselines in process mode, which share no lock at all, show the same loss (E_4 between 0.72 and 0.74) and scale by 2.50–2.67×. The loss is therefore a property of the testbed—four cores shared between the workers and the kernel's network stack—and process mode is the reference for what the hardware allows (Section 3). Thread-mode `uringpy` reaches that reference.

Third, every baseline loses throughput when given more threads. At four worker threads `asyncio` delivers 0.52× and 0.48× of one worker through its two APIs, and `uvloop` 0.59× and 0.36×, with 1.6 to 2.2 of the four cores busy and a 99th-percentile latency of 15–27 ms against 2.7–4.2 ms for the same engines in process mode. This is the pattern of `py-epoll` in Table 6; we did not count the baselines' GIL hand-offs. A loop in C does not by itself prevent it: `uvloop`'s loop is libuv, but it calls back into Python for every event, and it scales no better than `asyncio`.

In absolute terms, one `uringpy` worker is 1.44× faster than the fastest baseline worker (`uvloop-proto`), 1.50× faster than `asyncio-proto` and 2.11× faster than `asyncio`. At four workers the comparison that matters is with the baselines run the way they have to be run under the GIL, as processes: `uringpy` threads are 1.33× faster than `uvloop-proto`, 1.40× faster than `asyncio-proto` and 1.83× faster than `asyncio`. Against the baselines' threads, a configuration nobody would deploy, the factor is 7 to 10. It should be kept in mind that this workload favours the runtime as much as any can: the baselines execute Python for every event and `uringpy` executes none. (`uvloop` was measured in a separate experiment; see Section 7 on comparisons across experiments.)

Free-threaded build. The lower part of Table 8 repeats the experiment without the GIL. `asyncio` threads now scale, by 2.64 ± 0.08 and 2.50 ± 0.05, within 2–4% of the throughput of the same engines in process mode. The collapse in the upper part of the table is thus an effect of the GIL and not of `asyncio`'s design, and removing the GIL removes it. The price `asyncio` pays for the free-threaded build on one worker is small, about 4–5%. `uringpy` is unaffected by the build, as expected of a loop that does not enter the interpreter: its throughput at four workers on the two builds differs by a factor of 0.996 ± 0.005. What remains of its advantage is a difference in speed that narrows slightly as workers are added: 1.58× over `asyncio-proto` and 2.25× over `asyncio` at one worker, 1.46× and 1.96× at four. On the free-threaded build the contribution of the runtime is therefore speed per worker, not scaling.

A Python handler: one hand-off per request. In Table 9 every request calls a Python function that parses the request line, computes a checksum of the path and builds the response. On one worker the handler costs `uringpy` about 10% of its transport-only throughput in Table 8. With the GIL taken once per request (`uringpy-app`), worker threads stop scaling after the second: $S_2 = 1.53 \pm 0.02$ and $S_4 = 1.55 \pm 0.04$, with 3.24 of the four cores busy. Worker processes scale as on the transport path ($S_4 = 2.33 \pm 0.06$), so at four workers threads deliver 0.67 ± 0.02 of what processes deliver.

The bound (1) does not account for this, and the handler sweep of Table 10 shows what does. The handler holds the GIL for $t_p = 1.20$ µs per request on one worker, so $f = 0.15$ and $1/f \approx 6.8$: the bound allows more scaling than four cores can give. At four threads a request holds the GIL for 2.21 µs and waits 8.51 µs to get it. The GIL

Table 8: Transport path, 13-byte body: throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling $S_4 = \bar{x}_4/\bar{x}_1$; busy server cores C_N; throughput per busy core at four workers relative to one, $E_4 = (\bar{x}_4/C_4)/(\bar{x}_1/C_1)$; 99th-percentile latency at four workers.

Build	Engine	Workers as	N=1	N=2	N=3	N=4	S_4	C_1	C_4	E_4	p99$_4$ (ms)
GIL	`uringpy`	thread	140.7 ± 3.0	228.3 ± 6.0	286.5 ± 3.9	328.4 ± 0.8	2.33 ± 0.05	1.18	3.91	0.70	1.7
GIL	`uringpy`	process	144.4 ± 3.8	230.4 ± 7.0	288.5 ± 1.9	328.9 ± 1.2	2.28 ± 0.06	1.19	3.92	0.69	1.6
GIL	`asyncio`	thread	66.8 ± 1.1	47.8 ± 0.8	35.5 ± 0.1	34.4 ± 0.2	0.52 ± 0.01	1.07	1.76	0.31	15.4
GIL	`asyncio`	process	68.1 ± 1.0	116.8 ± 1.4	151.7 ± 2.7	179.6 ± 0.8	2.64 ± 0.04	1.08	3.91	0.73	2.7
GIL	`asyncio-proto`	thread	93.6 ± 1.6	95.2 ± 1.6	59.9 ± 0.5	45.0 ± 0.9	0.48 ± 0.01	1.12	2.17	0.25	16.7
GIL	`asyncio-proto`	process	94.3 ± 1.6	157.6 ± 4.2	201.2 ± 3.4	235.3 ± 1.7	2.50 ± 0.05	1.12	3.90	0.72	3.5
GIL	`uvloop`	thread	72.5 ± 1.0	–	–	43.0 ± 0.5	0.59 ± 0.01	1.09	1.69	0.38	19.9
GIL	`uvloop`	process	72.4 ± 1.9	–	–	193.6 ± 1.9	2.67 ± 0.07	1.09	3.92	0.74	4.2
GIL	`uvloop-proto`	thread	98.0 ± 2.3	–	–	35.0 ± 0.2	0.36 ± 0.01	1.13	1.59	0.25	26.6
GIL	`uvloop-proto`	process	96.5 ± 1.8	–	–	247.0 ± 0.6	2.56 ± 0.05	1.12	3.90	0.74	3.4
free-thr.	`uringpy`	thread	142.4 ± 5.1	229.7 ± 3.7	285.8 ± 5.3	327.0 ± 1.5	2.30 ± 0.08	1.19	3.91	0.70	1.8
free-thr.	`uringpy`	process	142.2 ± 5.1	228.0 ± 3.6	287.8 ± 2.3	325.2 ± 1.6	2.29 ± 0.08	1.19	3.92	0.69	1.7
free-thr.	`asyncio`	thread	63.2 ± 1.8	108.8 ± 0.6	141.0 ± 2.5	166.6 ± 0.9	2.64 ± 0.08	1.08	3.93	0.73	2.8
free-thr.	`asyncio`	process	64.9 ± 1.1	111.0 ± 1.6	146.1 ± 2.4	172.5 ± 2.6	2.66 ± 0.06	1.08	3.92	0.73	2.8
free-thr.	`asyncio-proto`	thread	90.0 ± 1.8	147.2 ± 2.8	189.8 ± 1.7	224.6 ± 1.2	2.50 ± 0.05	1.12	3.90	0.72	3.7
free-thr.	`asyncio-proto`	process	91.3 ± 1.1	151.7 ± 1.7	196.1 ± 1.5	230.4 ± 1.3	2.52 ± 0.03	1.12	3.92	0.72	3.6

`uvloop` was measured in a separate experiment at one and four workers only and is not installed in the free-threaded image.

utilisation is $U_4 = 0.41$: for 59% of each second the lock is not being used to run a handler, while workers wait for it. That wait is the whole difference between threads and processes. Per worker, a request takes $4/184.8\,\text{k} = 21.6\ \mu$s in thread mode and $4/289.5\,\text{k} = 13.8\ \mu$s in process mode; the difference, 7.8 μs, is the 8.5 μs of waiting less the 0.6 μs by which the hold is shorter in thread mode.

From (2) with $h = 1$, the cost of a hand-off is $c = 1/X_4 - t_{p,4}$ wherever the GIL is the limit. Across the six handler sizes of Table 10 this gives between 2.9 and 5.7 μs, with no trend in the size of the handler. A hand-off thus costs several microseconds whatever the handler does, and for a handler of one or two microseconds it is the hand-off, not the handler, that occupies the lock.

As the handler grows, f approaches 1 and the bound (1) becomes the binding constraint. With 300 added iterations ($t_p = 30.8\ \mu$s, $f = 0.82$) the bound is 1.23 and measured S_4 is 1.04 ± 0.03; with 3000 iterations the bound is 1.03 and the measurement 1.00 ± 0.01; the GIL utilisation is 0.92–0.98. In all 24 thread-mode cells of the table, S_2 and S_4 for both engines, the measured scaling is at or below the bound. Processes, by contrast, scale better as the handler grows, up to 3.98×, because a larger share of each request is then processor time in the worker itself and a smaller share is kernel network processing.

GIL batching: one hand-off per batch. Taking the GIL once for all requests found in one pass over the completion queue leaves f unchanged and reduces h from 1 to 0.022, one acquisition for about 45 requests. In Table 9, where the two variants are interleaved in one experiment, threads then scale by 2.12 ± 0.07 instead of 1.55 ± 0.04. Four threads deliver 275.6 thousand requests per second instead of 196.3 (1.40 ± 0.04), which is 0.93 ± 0.02 of what four processes deliver, and one worker is not slowed (1.03 ± 0.03). In Table 10 the wait per request at four threads falls from 8.51 to 1.60 μs. With 30 added iterations batching raises S_4 from 1.00 to 1.63 and the GIL utilisation from 0.50 to 0.92; with 100 iterations, from 1.11 to 1.24 and from 0.77 to 0.96. From 300 iterations upward it changes nothing, because (1) binds. Batching thus recovers what hand-offs cost and leaves the capacity of the lock itself: with 30 or more added iterations, where batched threads still fall well short of processes, the GIL is in use for more than 90% of the time. That capacity is somewhat lower than (1) predicts from one-worker hold times, because the hold time itself grows under load (4.00 to 5.92 μs with 30 iterations), as it does in process mode. The price of batching is that no response of a batch is queued until all its handlers have run, and with long handlers the interpreter's switch interval interrupts a batch.

Baselines with the handler. With the same handler, the baselines' threads fall to 0.52× (`asyncio-app`), 0.48× (`asyncio-proto-app`) and 0.35× (`uvloop-proto-app`) of one worker. The strongest baseline is `uvloop-proto-app` in process mode, at 219.9 thousand requests per second on four workers. Against it, `uringpy-app` threads with the GIL taken per request are slower (0.89 ± 0.02), and with GIL batching they are faster (1.25 ± 0.03); process against process, `uringpy-app` is 1.33 ± 0.03 times faster. On one worker `uringpy-app` is 1.49× faster than `uvloop-proto-app`, 1.56× faster than `asyncio-proto-app` and 2.11× faster than `asyncio-app`. The four-process cell of `asyncio-proto-app` varied by 9% between runs and is marked in the table.

Free-threaded build with the handler. Without the GIL there are no hand-offs to save: `uringpy-app` threads scale by 2.32 ± 0.03, the same as its processes (0.99 ± 0.01), and GIL batching makes no difference (1.00 ± 0.03). `asyncio-proto-app` threads scale by

Table 9: Application-handler workload: every request calls a Python handler. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, busy server cores and 99th-percentile latency at four workers.

Build	Engine	Workers as	N=1	N=2	N=4	S_2	S_4	C_4	p99$_4$ (ms)
GIL	`uringpy-app`	thread	126.3 ± 1.7	192.7 ± 1.6	196.3 ± 4.5	1.53 ± 0.02	1.55 ± 0.04	3.24	3.5
GIL	`uringpy-app`	process	125.8 ± 1.8	202.5 ± 3.5	292.7 ± 5.8	1.61 ± 0.04	2.33 ± 0.06	3.90	1.9
GIL	`uringpy-app-batch`	thread	130.1 ± 3.5	204.1 ± 2.6	275.6 ± 5.6	1.57 ± 0.05	2.12 ± 0.07	3.70	2.1
GIL	`uringpy-app-batch`	process	128.9 ± 2.7	206.4 ± 1.7	297.4 ± 0.8	1.60 ± 0.04	2.31 ± 0.05	3.92	1.8
GIL	`asyncio-app`	thread	59.9 ± 1.2	40.2 ± 0.4	31.2 ± 0.5	0.67 ± 0.02	0.52 ± 0.01	1.69	17.1
GIL	`asyncio-app`	process	60.2 ± 1.8	104.7 ± 1.3	158.7 ± 9.5	1.74 ± 0.06	2.64 ± 0.18	3.93	3.4
GIL	`asyncio-proto-app`	thread	80.7 ± 0.8	64.1 ± 0.9	38.6 ± 0.4	0.79 ± 0.01	0.48 ± 0.01	1.91	18.8
GIL	`asyncio-proto-app`	process	81.0 ± 1.7	137.5 ± 1.9	187.1 ± 21.0	1.70 ± 0.04	2.31 ± 0.26[†]	3.92	5.8
GIL	`uvloop-proto-app`	thread	84.6 ± 2.1	68.9 ± 0.7	29.9 ± 0.3	0.81 ± 0.02	0.35 ± 0.01	1.46	33.3
GIL	`uvloop-proto-app`	process	85.2 ± 1.5	143.2 ± 2.4	219.9 ± 1.5	1.68 ± 0.04	2.58 ± 0.05	3.92	3.7
free-thr.	`uringpy-app`	thread	125.3 ± 1.5	199.2 ± 6.4	290.5 ± 2.3	1.59 ± 0.05	2.32 ± 0.03	3.91	1.9
free-thr.	`uringpy-app`	process	125.8 ± 1.8	202.4 ± 2.5	293.8 ± 1.9	1.61 ± 0.03	2.34 ± 0.04	3.91	1.8
free-thr.	`uringpy-app-batch`	thread	127.1 ± 3.1	204.1 ± 7.9	290.9 ± 6.8	1.61 ± 0.07	2.29 ± 0.08	3.90	2.0
free-thr.	`uringpy-app-batch`	process	129.6 ± 4.4	208.6 ± 3.6	296.4 ± 1.6	1.61 ± 0.06	2.29 ± 0.08	3.91	1.8
free-thr.	`asyncio-app`	thread	56.5 ± 0.8	96.8 ± 0.6	151.1 ± 2.3	1.71 ± 0.03	2.68 ± 0.05	3.93	3.2
free-thr.	`asyncio-app`	process	56.9 ± 0.5	98.8 ± 2.9	155.2 ± 2.2	1.74 ± 0.05	2.73 ± 0.05	3.91	3.1
free-thr.	`asyncio-proto-app`	thread	78.6 ± 1.0	129.2 ± 0.4	198.3 ± 1.2	1.64 ± 0.02	2.52 ± 0.04	3.92	4.2
free-thr.	`asyncio-proto-app`	process	79.4 ± 1.7	134.2 ± 1.8	204.5 ± 3.2	1.69 ± 0.04	2.58 ± 0.07	3.90	4.0

[†] A configuration in this row had a coefficient of variation above 5%.

2.52 ± 0.04, and the gap between the two at four workers is 1.46 ± 0.01. On the GIL build, batched threads reach 0.95 of the throughput of free-threaded threads.

The free-threaded column of Table 10 carries a warning for longer handlers, however. Thread scaling there falls as the handler grows, to 1.20 with 300 added iterations and 1.01 with 3000, no better than under the GIL, and the time per handler call grows from 32 μs on one worker to 123 μs on four. A control without the runtime shows the same behaviour: plain Python threads calling the handler with 300 iterations complete 29.7, 42.5 and 30.3 thousand calls per second with one, two and four threads. The effect therefore belongs to this interpreter version and this handler, an interpreted integer loop, and not to the runtime. We did not identify its cause, and it should not be read as a general statement about the free-threaded build; it does mean that the free-threaded results of Table 9 hold for the short handler measured there.

Response size. Table 11 varies the response body with four worker processes on the GIL build, against `asyncio`'s Protocol API. Up to 1 KiB `uringpy` is 1.4× faster. At 16 KiB both engines deliver 119 thousand requests per second, and from there on they are indistinguishable: the client receives 15.7–15.8 Gbit/s whatever the body size and whichever the engine. The network path is the limit, and those rows measure it and not the servers. As responses grow, each is sent in several pieces and batches shrink, so `uringpy`'s system calls per request rise from 0.02 to 2.4. The runtime's advantage in requests per second is thus confined to responses small enough for per-request overhead, not data transfer, to dominate; on this network that means below 16 KiB. We did not determine what sets the limit of the path.

6 Discussion

Attributing a speed-up. A Python loop on `io_uring` that is faster than `asyncio` has not thereby shown that `io_uring` is the reason. On one worker the interface is worth a factor of 1.14–1.25, the saving of system calls is exhausted at 16 completions per call, and a bare loop in C is only 1.05–1.15× faster than the same loop in Python. A figure of several hundred completions per system call, though real, says little about single-worker throughput. Most of the distance between one `asyncio` worker and one `uringpy` worker (2.11×) comes from what a full event loop does per event, not from the interface it waits on nor from the language its dispatch is written in.

What batching buys. For worker threads under the GIL, the larger benefit of a completion interface is that the lock can be given up once per batch instead of once per system call. That benefit is available to a loop written in Python: `py-uring` is within 10% of the C reactor at four workers. It is also easily forfeited: holding the GIL across the one submitting call, in which the kernel does the sends, returns the same loop to the throughput of a single worker. An `asyncio`-compatible event loop on `io_uring` could in principle be in either regime, depending on where it releases the GIL and on how much Python it runs per event. We did not measure the existing ones.

Three ways to keep hand-offs rare. The runtime offers a reactor that runs no Python, a reactor that calls a handler under one GIL acquisition per batch, and a batch interface for loops written in Python. On the transport path the first is 1.11× faster than the third at four workers, so its advantage over a well-behaved Python loop is modest. With a handler, the second turns a design that loses to the best process-per-worker baseline (0.89×) into one that beats it (1.25×) while keeping one interpreter. We did not measure a Python loop that also calls the

Table 10: Handler work swept (iterations of an interpreted loop added to every request), GIL build unless stated, with the GIL taken once per request (`uringpy-app`) or once per batch of requests (`uringpy-app-batch`). One-worker throughput $\bar{x}_1$ in thousands of requests per second (mean ± 95% half-width, $n = 5$); GIL hold time per request t_p at one worker; $f = t_p\,\bar{x}_1$; the bound $\min(4, 1/f)$ of (1); measured scaling of worker threads (S_2, S_4), of worker processes (S_4^{proc}) and of threads on the free-threaded build (S_4^{ft}); and, at four worker threads, GIL hold time $t_{p,4}$ and wait time w_4 per request, GIL acquisitions per request, and GIL utilisation $U_4 = \bar{x}_4\,t_{p,4}$.

GIL per	Work	$\bar{x}_1$	t_p (μs)	f	bound	S_2	S_4	S_4^{proc}	S_4^{ft}	$t_{p,4}$	w_4	acq.	U_4
request	0	123.2 ± 1.8	1.20	0.15	4.00	1.54 ± 0.05	1.50 ± 0.03	2.36 ± 0.13	2.12 ± 0.43†	2.21	8.51	1.000	0.41
request	30	91.3 ± 1.1	4.11	0.38	2.66	1.46 ± 0.04	1.00 ± 0.02	2.64 ± 0.03	2.14 ± 0.34†	5.42	26.36	1.000	0.50
request	100	55.4 ± 1.3	11.09	0.62	1.63	1.23 ± 0.05	1.11 ± 0.03	3.08 ± 0.08	1.70 ± 0.04	12.61	43.09	1.000	0.77
request	300	26.5 ± 0.2	30.75	0.82	1.23	1.05 ± 0.05	1.04 ± 0.03	3.51 ± 0.03	1.20 ± 0.02	33.38	103.26	1.000	0.92
request	1000	8.62 ± 0.38	106.23	0.92	1.09	1.05 ± 0.05	0.98 ± 0.05	3.92 ± 0.03	1.07 ± 0.07	113.08	348.42	1.000	0.95
request	3000	3.07 ± 0.03	315.80	0.97	1.03	0.98 ± 0.08†	1.00 ± 0.01	3.98 ± 0.07	1.01 ± 0.07†	320.40	969.16	1.000	0.98
batch	0	126.6 ± 2.0	1.24	0.16	4.00	1.61 ± 0.04	2.20 ± 0.04	–	–	2.42	1.60	0.021	0.67
batch	30	96.2 ± 0.9	4.00	0.38	2.60	1.54 ± 0.07	1.63 ± 0.03	–	–	5.92	12.27	0.021	0.92
batch	100	57.8 ± 0.2	10.87	0.63	1.59	1.33 ± 0.02	1.24 ± 0.08†	–	–	13.44	36.70	0.020	0.96
batch	300	27.1 ± 0.4	30.51	0.83	1.21	1.09 ± 0.05	1.04 ± 0.04	–	–	35.02	102.09	0.020	0.98
batch	1000	8.99 ± 0.12	102.44	0.92	1.09	1.02 ± 0.06	1.00 ± 0.02	–	–	212.83	222.60	0.021	n/a
batch	3000	3.14 ± 0.07	310.27	0.97	1.03	1.00 ± 0.02	0.98 ± 0.02	–	–	978.66	306.73	0.022	n/a

$t_{p,4}$ and w_4 in μs. The per-request and per-batch rows come from separate experiments. n/a: with batching and long handlers the interpreter's switch interval interrupts a batch, so the recorded hold includes time waiting to resume. † A configuration in this ratio had a coefficient of variation above 5%.

Table 11: Response-body size (four worker processes, GIL build). Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$); for `uringpy` also the data rate received by the client, busy server cores and system calls per request.

Body	`uringpy`	`asyncio-proto`	Gbit/s	C	sys/req
64 B	327.9 ± 0.9	234.9 ± 1.6	0.4	3.91	0.02
1 KiB	321.0 ± 1.6	230.7 ± 1.5	2.9	3.92	0.02
16 KiB	119.3 ± 0.5	118.9 ± 0.7	15.7	3.32	0.64
64 KiB	30.0 ± 0.1	30.0 ± 0.1	15.8	1.45	0.74
256 KiB	7.49 ± 0.04	7.50 ± 0.01	15.8	1.29	1.23
1 MiB	1.87 ± <0.01	1.87 ± 0.01	15.8	1.61	2.44

handler, which would be the direct comparison for the second.

The limit that remains. Hand-offs are an overhead that design can remove; the time the handler itself holds the GIL is not. Once a request spends more than about half of its processor time in Python, threads under the GIL add little whatever the design, and processes or the free-threaded build are needed. On the free-threaded build the runtime keeps a factor of 1.5–2× in speed for short handlers, and our one longer handler did not scale there either, for reasons that lie in the interpreter.

Threads or processes. Since worker processes reach the same throughput as `uringpy`'s worker threads, the case for threads is not speed. It is that the workers live in one interpreter and can share state without inter-process communication. We did not measure a workload that depends on shared state, so this paper establishes that the option exists at little or no cost in throughput, not how much a given application gains from it.

7 Threats to Validity

Table 12 lists the ways a measurement of this kind can mislead and what was done about each. The limits that remain are these.

Testbed. The server has four cores, so scaling beyond four workers is not shown, and at four workers the kernel's network processing competes with the workers for the same cores; E_4 in Table 8 quantifies that. Whether the ordering of Table 6 persists with more cores, where more threads compete for the GIL, is not known. The machines are cloud virtual machines of one family running one kernel and one CPython version (3.14.7), all runs were made on one pair of machines within two days, and the free-threaded results describe that version of an implementation that is still changing. Above 16 KiB responses the network path, not the server, was the limit.

Workload. The workloads are one protocol exchange with a fixed or lightly computed response; requests are not parsed, and TLS, request bodies and heavier application logic are not covered. The transport workload runs no Python in `uringpy` and Python on every event in the baselines, which is the most favourable case for the runtime. Load is 400 keep-alive connections from a closed-loop generator and nothing else: batch sizes, and with them both the saving of system calls and the number of GIL hand-offs per request, depend on how many completions are waiting, so the effects reported here may be smaller under light load. `io_uring` is used in its basic form (Section 4); features such as multishot receive or deferred task running were not evaluated. The `asyncio` and `uvloop` transports set `TCP_NODELAY`; the C reactors and the Python loops do not, which we expect to be immaterial for responses of one segment but did not test. The Python loops of the 2×2 design are bare loops, so

the loop-placement effect measured on one worker is a lower bound for a full event loop.

What was not measured. GIL hand-offs were counted for our own engines and not for `asyncio` or `uvloop`. The batch cap was varied only for the C reactor on one worker, not for the configurations in which it would change the number of hand-offs. The hand-off cost c is derived from throughput and hold time, not timed directly, and a single value of it does not predict every cell: it ranges from 2.9 to 5.7 μs. The handler is one small function, optionally lengthened by an interpreted loop; the slowdown of that loop across threads on the free-threaded build was reproduced without the runtime but not explained. The `io_uring` event loops that motivate the study were not run.

Measurement. Latency is measured by a closed-loop generator and is not a tail-latency estimate; the latency columns are means of per-run 99th percentiles. The contention probe is reliable only for one competing thread on this machine (Section 3). Runs with the two longest handlers reported socket errors for up to 0.4% of requests. Seven of the 227 configurations had a coefficient of variation above 5%; those that enter a reported ratio are marked. One test of the runtime's test suite failed once on the free-threaded image and passed in nine further runs; we have not identified it.

Comparisons across experiments. Repetitions are interleaved within an experiment, not between experiments. One `uringpy` worker was measured in three of them, at 140.7 ± 3.0, 140.3 ± 4.3 and 144.0 ± 4.5 thousand requests per second, a spread of 3%. Ratios between configurations of one experiment are unaffected. Comparisons that span two experiments carry that additional uncertainty, and their intervals, where given, reflect only the variation within each: those involving `uvloop` on the transport path, the capped `io_uring` loop against `epoll`, `py-epoll` against `asyncio`, the cost of the handler on one worker, the two engines of Table 10, and every comparison of the GIL build with the free-threaded build. The experiments were also run at three versions of the code as it was revised; each result folder records its commit and image, and the reactor's transport path gave the same four-worker throughput before and after (328.4 ± 0.8 and 329.0 ± 0.8).

8 Conclusion

For a Python server on `io_uring`, what batching buys depends on how many threads share the interpreter. For one worker it saves system calls, which together with the interface is worth a factor of 1.14–1.25 and is exhausted by about 16 completions per call. For several worker threads under the GIL it does something more valuable: it lets the lock be handed over once per batch instead of once per system call or once per request. In our measurements that, and not the language of the loop, separates designs that scale from designs that lose throughput. A Python loop that gives up the GIL twice per batch scales by 2.21× on four cores, a C reactor that takes it once per request by 1.55×, and a Python loop that releases it twice per request falls to 0.64×; the first of these falls to 0.98× if it merely holds the GIL during its submitting call. A runtime built on this observation gives one interpreter on a standard build the throughput of a process per worker when no Python runs per request, and, by taking the GIL once per batch, 0.93 of it when a short Python handler does, exceeding the best process-per-worker baseline in both cases. The advantage has limits, which we measured. Once the handler itself holds the GIL for most of a request, no design recovers thread scaling under the GIL. On the free-threaded build `asyncio` threads scale as well and a factor of 1.5–2× in speed remains. With responses of 16 KiB or more, the network and not the server was the limit. The GIL, not the system call, is what batching has to amortize.

Acknowledgment

[[PENDING: Author to confirm the wording before submission. IEEE requires content generated by AI to be disclosed in this section.]] The author used Claude (Anthropic), an AI assistant, in revising this work: to review an earlier draft and propose additional experiments in response; to write parts of the implementation (the application-handler reactor, GIL batching, the `epoll` reactor, the benchmark driver, the table generator and the tests) and of the documentation; and to draft text in every section of this paper, including the abstract. The author ran all experiments, reviewed and tested this content, and is responsible for the paper's claims.

Code and Data Availability

The runtime and all benchmark scripts are open source (MIT) at `https://github.com/marimuthuvm/uringpy` and released on PyPI (`pip install uringpy`). The repository contains the raw output of every run reported here (`benchmarks/results/`), the script that generates the result tables from it (`benchmarks/make_tables.py`), the container recipe for both interpreters, and a description of each experiment (`benchmarks/EXPERIMENTS.md`). **[[PENDING: Archive the release that contains these results and cite its DOI here; the earlier archive,** `https://doi.org/10.5281/zenodo.23129560`**, predates them.]]**

Conflict of Interest

The author has no financial or personal relationships that could have influenced this work, and declares no conflict of interest. No external funding was received.

Table 12: Pitfalls and how each is handled.

Pitfall	What is done	Evidence
The load generator, not the server, is the limit	The client has twice the server's cores; client processor use is recorded for every run and configurations above 85% are flagged	Section 5.5; raw data
Hyper-threads counted as cores	Machine family with one vCPU per physical core	Table 2
Noise mistaken for an effect	Five repetitions, interleaved in shuffled order, a fresh server per run, 95% confidence intervals; cross-experiment spread stated	every results table
A weak baseline	`asyncio` through both its streams and its faster Protocol API, and `uvloop`, with and without the handler; process mode as well as thread mode; one interpreter version for every engine	Tables 8, 9
GIL and free-threaded builds differing in more than the GIL	Both interpreters built from one recipe; the GIL state reported by the server is recorded per run	Table 2
Crediting `io_uring` for what C code does, or the reverse	The 2×2 design varies the two separately; the batch cap varies batching alone	Tables 4–7
Crediting the loop's language for what its handling of the GIL does	A variant of the Python loop that differs only in holding the GIL while submitting; GIL releases and acquisitions counted	Table 6
System-call counts estimated, not measured	Both C reactors count their system calls and completed requests	Tables 4, 7
A one-worker baseline that also uses other cores	Server processor use recorded; busy cores and throughput per busy core reported next to scaling	Table 8
A model fed with assumed numbers	GIL hold time, wait time and acquisitions measured per request; the bound compared with measured scaling at six handler sizes	Table 10
Engines doing different protocol work	Every engine answers each `recv` with identical bytes; none parses HTTP	Section 4
Errors hidden in a throughput figure	Socket errors, non-2xx responses, handler errors and failed runs recorded and reported	Section 5.5; raw data
Numbers not traceable to code	Commit, image, interpreter and seed recorded per experiment; raw runs archived, superseded runs included; tables generated from them	repository

References

[1] J. Axboe, "Efficient IO with io_uring," 2019. [Online]. Available: `https://kernel.dk/io_uring.pdf`

[2] MagicStack, "uvloop: Ultra fast asyncio event loop." [Online]. Available: `https://github.com/MagicStack/uvloop`

[3] L. Soares and M. Stumm, "FlexSC: Flexible system call scheduling with exception-less system calls," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010.

[4] S. Han, S. Marinos, S. Shenker, and S. Ratnasamy, "MegaPipe: A new programming interface for scalable network I/O," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012.

[5] E. Jeong, S. Woo, M. Jamshed, H. Jeong, S. Ihm, D. Han, and K. Park, "mTCP: A highly scalable user-level TCP stack for multicore systems," in *Proc. 11th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2014.

[6] A. Belay, G. Prekas, A. Klimovic, S. Grossman, C. Kozyrakis, and E. Bugnion, "IX: A protected dataplane operating system for high throughput and low latency," in *Proc. 11th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2014.

[7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2022, doi: 10.1145/3534056.3534945.

[8] M. Jasny, M. El-Hindi, T. Ziegler, V. Leis, and C. Binnig, "High-performance DBMSs with io_uring: When and how to use it," *Proc. VLDB Endow.*, vol. 19, no. 9, 2026.

[9] G. Banga, J. C. Mogul, and P. Druschel, "A scalable and explicit event delivery mechanism for UNIX," in *Proc. USENIX Annu. Technical Conf.*, 1999.

[10] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in *Proc. 18th ACM Symp. Operating Systems Principles (SOSP)*, 2001.

[11] G. van Rossum, "PEP 3156: Asynchronous IO support rebooted: the asyncio module," Python Enhancement Proposals, 2012.

[12] A. Pesterev, J. Strauss, N. Zeldovich, and R. T. Morris, "Improving network connection locality on multicore systems," in *Proc. 7th ACM European Conf. Computer Systems (EuroSys)*, 2012.

[13] M. Kerrisk, "The SO_REUSEPORT socket option," *LWN.net*, Mar. 2013. [Online]. Available: `https://lwn.net/Articles/542629/`

[14] P. Enberg, A. Rao, and S. Tarkoma, "The impact of thread-per-core architecture on application tail latency," in *Proc. ACM/IEEE Symp. Architectures for Networking and Communications Systems (ANCS)*, 2019.

[15] D. Beazley, "Understanding the Python GIL," presented at PyCon, Atlanta, GA, USA, 2010. [Online]. Available: `http://www.dabeaz.com/GIL/`

[16] R. Odaira, J. G. Castanos, and H. Tomari, "Eliminating global interpreter locks in Ruby through hardware transactional memory," in *Proc. 19th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2014, doi: 10.1145/2555243.2555247.

[17] S. Gross, "PEP 703: Making the Global Interpreter Lock Optional in CPython," Python Enhancement Proposals, 2023.

[18] T. Wouters, M. Page, and S. Gross, "PEP 779: Criteria for supported status for free-threaded Python," Python Enhancement Proposals, 2025.

[19] E. Snow, "PEP 684: A Per-Interpreter GIL," Python Enhancement Proposals, 2022.

[20] E. Snow, "PEP 734: Multiple Interpreters in the Stdlib," Python Enhancement Proposals, 2023.

[21] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, 2011.

[22] K. Aditya, "Scaling asyncio on free-threaded Python," Quansight Labs, Sep. 2025. [Online]. Available: `https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python`

[23] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[24] "uringloop: An io_uring event loop for asyncio." [Online]. Available: `https://github.com/thearchitector/uringloop`

[25] "uringcore: High-performance asyncio event loop using io_uring." [Online]. Available: `https://github.com/ankitkpandey1/uringcore`

[26] "Granian: A Rust HTTP server for Python applications." [Online]. Available: `https://github.com/emmett-framework/granian`

[27] "Gunicorn: Python WSGI HTTP server for UNIX." [Online]. Available: `https://github.com/benoitc/gunicorn`

[28] "Uvicorn: An ASGI web server for Python." [Online]. Available: `https://github.com/encode/uvicorn`

[29] "Netty incubator transport for io_uring." [Online]. Available: `https://github.com/netty/netty-incubator-transport-io_uring`

[30] "tokio-uring: An io_uring backed runtime for Rust." [Online]. Available: `https://github.com/tokio-rs/tokio-uring`

[31] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, doi: 10.1145/1297027.1297033.

[32] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, doi: 10.1145/1508244.1508275.

[33] A. Uta, A. Custura, D. Duplyakin, I. Jimenez, J. Rellermeyer, C. Maltzahn, R. Ricci, and A. Iosup, "Is big data performance reproducible in modern cloud networks?," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020.

[34] W. Glozer, "wrk: Modern HTTP benchmarking tool." [Online]. Available: `https://github.com/wg/wrk`

[35] G. Tene, "wrk2: A constant throughput, correct latency recording variant of wrk." [Online]. Available: `https://github.com/giltene/wrk2`

[36] S. Eyerman and L. Eeckhout, "Modeling critical sections in Amdahl's law and its implications for multicore design," in *Proc. 37th Annu. Int. Symp. Computer Architecture (ISCA)*, 2010.

[37] Google Cloud, "CPU platforms," Compute Engine documentation. [Online]. Available: `https://docs.cloud.google.com/compute/docs/cpu-platforms`

Marimuthu Velayutham is an independent researcher working on high-performance systems software and language runtimes, with interests in asynchronous I/O, operating-system interfaces, and the performance of managed-language runtimes.